Enterprise Intelligence Portal : Architecture & Strategy Guide

Review of World View Dashboard for AIB-AXYS Africa Research Dpt, Trading Dpt, and Leadership

Author: Samuel Mburu Kinuthia

02/03/2026

Table of Contents

Chapter 1: Architectural & Compliance Audit *7*

1.1 Executive Summary **7**

1.2 Detailed Site Brief **7**

1.2.1 Core Functionality 7

1.2.2 Tech Stack 7

1.2.3 Architecture & Data Flow 7

1.3 API Audit (Existing) **8**

1.3.1 Free / Open / Public APIs 8

1.3.2 Paid / Commercial / Rate-Limited APIs 8

1.4 Missing APIs for Enterprise Production **8**

1.5 Copyright, Licensing & Compliance **9**

1.5.1 Codebase License 9

1.5.2 Compliance Risk: Internal Corporate Use is Commercial Use 9

1.5.3 Recommendations for Safe Enterprise Replication 9

Chapter 2: Security & Authentication Blueprint *10*

2.1 Executive Summary **10**

2.2 Security Audit & Red Flags **10**

2.2.1 Server-Side Request Forgery (SSRF) Risks 10

2.2.2 Insecure Session Handling & Scraping 10

2.2.3 Weak API Gateway Protection & Network Perimeter 10

2.2.4 Tauri Desktop IPC Boundaries 10

2.3 Tech Stack Recommendations for Enterprise Identity **11**

2.3.1 Recommended Solution: Auth0 or Microsoft Entra ID 11

2.4 Authentication Implementation Steps **11**

Step 1: IdP Provisioning....11
Step 2: Frontend Integration....11
Step 3: Secure the UI (Absolute Lockout)....11
Step 4: Edge API Protection....11
Step 5: Backend Authorization....12
Chapter 3: Data Specialization Strategy....13
3.1 Executive Summary....13
3.2 Backend & Pipeline Modifications....13
3.2.1 Spatial Resolution & Mapping (Deck.gl)....13
3.2.2 Database & Caching Adjustments....13
3.2.3 Ingestion Logic Overhaul....13
3.3 Regional API Integrations....14
3.3.1 Economic & Market Intelligence....14
3.3.2 Geographic, Infrastructure & Environment....14
3.3.3 News, Unrest & Geopolitics....14
3.4 Implementation Priorities....15
Chapter 4: UI/UX & Search Implementation Guide....16
4.1 Executive Summary....16
4.2 Summarized Custom Dashboard (UI/UX)....16
4.2.1 Design Philosophy: The "Dual-Lens" Approach....16
4.2.2 Frontend Stack Recommendations....16
4.2.3 Dashboard Architecture....16
4.3 Advanced Search Implementation....16
4.3.1 Search Engine Recommendations....16
4.3.2 The Unified Command Palette....17

4.3.3 Data Pipeline to Search Engine 17

Chapter 5: System Infrastructure Diagram *18*

5.1 Architectural Topology **18**

5.2 Core Data Flow Summary **20**

Chapter 6: Codebase Navigation Guide *21*

6.1 Executive Summary **21**

6.2 Core Directory Structure **21**

6.3 Where to Find What **21**

6.3.1 UI Components and Layouts 21

6.3.2 API Fetchers and Serverless Functions 22

6.3.3 State Management and Caching Logic 22

6.3.4 Configurations (Environment, Routing, Themes) 22

6.3.5 Data Definitions and Schemas 22

Chapter 7: Draft Development Guide: Architecture & UI *24*

7.1 Executive Summary **24**

7.2 Authentication & Database Architecture **24**

7.2.1 ORM and Database Selection 24

7.2.2 Identity via NextAuth / Auth.js 24

7.2.3 Proposed Database Schema 24

..... **25**

7.3 The "Easy-on-the-Eye" Dynamic Dashboard **25**

7.3.1 UI/UX Layout Strategy 25

7.3.2 Frontend State Management 25

7.4 Security Patching & Hardening **26**

7.4.1 Mitigating SSRF in Proxy Endpoints 26

7.4.2 Securing API Gateways 26

7.4.3 Robust Rate Limiting 26

Chapter 8: Draft Deployment Guide: Bulletproof CI/CD 27

8.1 Executive Summary 27

8.2 Modern Infrastructure Proposal 27

8.2.1 Why PM2 over Docker? 27

8.2.2 Architecture Overview 27

8.3 Step-by-Step Server Provisioning 27

Step 1: System Updates and Dependencies 27

Step 2: Install Node.js and PM2 27

Step 3: Clone the Repository 28

Step 4: Configure the PM2 Ecosystem 28

Step 5: Configure Nginx Reverse Proxy 28

8.4 The Automation Script (`deploy.sh`) 29

8.4.1 Rollback Procedure 30

Chapter 9: AI Execution Prompts for Developers 31

9.1 Executive Summary 31

9.2 Phase 1: Database and Authentication Initialization 31

9.3 Phase 2: Database Schema Generation 31

9.4 Phase 3: Dynamic Dashboard Construction 32

9.5 Phase 4: Security Hardening 32

Chapter 10: Data Pipeline & ETL Strategy 34

10.1 Executive Summary 34

10.2 Background Worker Architecture 34

10.2.1 Recommended Orchestrator: Inngest (or Trigger.dev) 34

10.3 Multi-Tier Caching & Database Strategy 34

10.3.1 The Ingestion Flow (Write Path) 34

10.3.2 The Presentation Flow (Read Path) 35

10.4 AI Execution Prompts for Developers 35

Chapter 11: QA & Testing Playbook 37

11.1 Executive Summary 37

11.2 Core Testing Strategy 37

11.2.1 Unit Testing Layer (Vitest / Jest) 37

11.2.2 End-to-End Testing Layer (Playwright) 37

11.3 Playwright Integration: Mocking Authentication 37

11.3.1 Network Interception Strategy 37

11.3.2 Example Playwright Setup Pattern 38

11.4 AI Execution Prompts for Developers 38

Chapter 12: Monitoring & Disaster Recovery Guide 40

12.1 Executive Summary 40

12.2 Application Monitoring Strategy 40

12.2.1 Local Logging: Pino 40

12.2.2 Server-Level Observability: PM2 40

12.2.3 Error Tracking: Sentry 40

12.3 Database Backup Protocol 40

12.3.1 Automated Backup Script 41

12.3.2 Cron Job Setup 42

12.4 Disaster Recovery: 3-Step Rollback Protocol 42

Step 1: Revert the Application Code 42

Step 2: Restore the Database 42

Step 3: Relaunch and Verify 43

Chapter 13: Project Management: Timeline, AI Impact & Costs 44

13.1 Executive Summary 44

13.2 Development Timeline & Complexity 44

Phase 1: Database & Identity Migration (Weeks 1-2) 44

Phase 2: Security & ETL Pipeline Orchestration (Weeks 3-4) 44

Phase 3: African UI Customization & Mapping (Weeks 5-7) 44

Phase 4: QA, Testing & Production Deployment (Weeks 8-9) 44

13.3 The AI Factor (Speed vs. Limitations) 45

13.3.1 Acceleration: Where AI Excels 45

13.3.2 Limitations: Where Human Developers Must Lead 45

13.4 FinOps & API Strategy 45

13.4.1 Infrastructure Cost Benefits 45

13.4.2 Low-Cost Mapping Strategy 45

13.4.3 Bloomberg Subscription & Financial APIs 46

13.5 Project Timeline (Gantt Chart) 46

Chapter 1: Architectural & Compliance Audit

1.1 Executive Summary

This chapter provides a foundational analysis of the "worldmonitor" platform. It serves as a roadmap for adapting the platform into a secure, internal enterprise dashboard for proprietary corporate use (e.g., by traders, researchers, and leadership). The analysis places a strong emphasis on system architecture, data integrations, and the legal/compliance risks associated with commercial data aggregation.

1.2 Detailed Site Brief

1.2.1 Core Functionality

WorldMonitor is an advanced, real-time global dashboard and data aggregation platform. It tracks and visualizes multi-domain geopolitical, environmental, economic, and military data on a unified interactive map. Core features include:

- **Live Event Tracking:** Aggregates global conflicts, unrest, natural disasters, and infrastructure outages.
- **Asset Monitoring:** Tracks military flights, maritime vessels (AIS), and satellite-detected thermal anomalies.
- **Economic & Market Intelligence:** Visualizes stock indices, ETF flows, critical minerals, and supply chain chokepoints.
- **AI/ML Summarization:** Uses client-side or edge-based machine learning to classify threats, analyze sentiment, and summarize news/intelligence feeds.
- **Multi-Platform Access:** Available as a web application and a native desktop application.

1.2.2 Tech Stack

The platform utilizes a modern, highly decoupled, and performant stack:

- **Frontend (Web):** TypeScript, Vite, React, Deck.gl (for high-performance WebGL map rendering), and standard CSS.
- **Desktop App:** Tauri (Rust-based), allowing the web app to run natively with a minimal footprint.
- **Backend / Edge Layer:** Node.js/Edge functions operating as a proxy/caching layer.
- **API Protocol:** Protocol Buffers (Protobuf) compiled via buf, indicating a highly typed, low-bandwidth RPC architecture.
- **State & Storage:** Redis for aggressive edge caching, and Convex for real-time state sync.
- **Local AI Processing:** Web Workers and Ollama integration for local LLM inference, keeping proprietary data from leaking to public LLMs.

1.2.3 Architecture & Data Flow

The architecture is designed as a read-heavy, aggregate-and-cache system:

1. **Ingestion (Backend/Cron):** Edge functions and scheduled scripts scrape RSS feeds, query external APIs, and process raw data.
2. **Caching (Edge):** Data is aggressively cached in Redis to serve high loads quickly.

3. **Transmission (RPC):** Data is serialized into Protobufs and sent to the client via strictly typed RPC endpoints.
4. **Rendering (Client):** The frontend receives Protobuf payloads, hydrates local state, and visualizes the data on a Deck.gl map canvas.
5. **Local ML (Client/Desktop):** Heavy computational tasks are offloaded to the browser via Web Workers or to the local desktop environment via Tauri/Ollama.

1.3 API Audit (Existing)

The platform aggregates a massive amount of external data. Based on the service definitions, the API breakdown is as follows:

1.3.1 Free / Open / Public APIs

- **GDELT Project:** Global database of society/news.
- **FRED (Federal Reserve Economic Data):** Macroeconomic signals.
- **World Bank:** Economic indicators.
- **BIS (Bank for International Settlements):** Credit and policy rates.
- **EONET (NASA):** Earth observatory natural events.
- **USGS:** Earthquake seismology data.
- **GDACS:** Global disaster alerts.
- **UCDP:** Uppsala Conflict Data Program.
- **Oref:** Israel Home Front Command alerts.

1.3.2 Paid / Commercial / Rate-Limited APIs

(Note: While some offer free tiers, corporate/internal use usually requires paid agreements)

- **OpenSky Network:** Aviation tracking (Free for academic/hobby; strictly paid for commercial/corporate).
- **ACLED (Armed Conflict Location & Event Data):** Highly restricted. Requires paid licensing for corporate application.
- **AIS/Maritime Data:** Tracking vessels requires premium providers (e.g., Spire, MarineTraffic).
- **Financial Market Data:** Stock indices, crypto, and ETF flows require enterprise licenses (e.g., Finnhub, AlphaVantage, Bloomberg APIs).
- **Map Tiles:** Mapbox, MapTiler, or similar providers are required for Deck.gl base maps.

1.4 Missing APIs for Enterprise Production

To transition this open-source dashboard into a secure, proprietary internal tool, the following infrastructure must be added:

- **Enterprise Identity & Access Management (IAM):** (e.g., Microsoft Entra ID / Azure AD, Okta, Auth0). Access must be restricted strictly to corporate employees via Single Sign-On (SSO).
- **Proprietary Data Connectors:** Integration with internal databases, trading systems, or premium terminal APIs (e.g., Bloomberg B-PIPE, Refinitiv) to fuse internal insights with global data.
- **Application Performance Monitoring (APM):** (e.g., Datadog, Splunk) to track system health and ensure high availability.
- **Managed Vector Database:** (e.g., Pinecone, self-hosted Milvus). Required to perform Retrieval-Augmented Generation (RAG) over proprietary research reports alongside global news.

1.5 Copyright, Licensing & Compliance

Several critical legal vectors must be considered, even for internal corporate use.

1.5.1 Codebase License

- A thorough review of the specific text of the LICENSE file in the root directory is necessary.
- If the license is MIT or Apache 2.0, the codebase can be forked, modified, and used internally without issue.
- If the license is GPL or AGPL (Copyleft), internal corporate use generally does not trigger the requirement to release source code, as the software is not "distributed" to the public. However, if external clients or partners access the web app, AGPL provisions may be triggered.

1.5.2 Compliance Risk: Internal Corporate Use is Commercial Use

A common misconception is that "internal use" equates to "non-commercial." From a data provider's perspective, internal use by a trading or research firm constitutes strictly commercial use.

- **ACLED Data:** Using ACLED data internally to inform trading or corporate strategy requires a commercial license.
- **OpenSky / Aviation Data:** The free tier explicitly prohibits internal corporate tracking.
- **Scraping Liability:** The codebase includes proxies and scrapers. Systematically scraping data to feed internal trading algorithms can violate Terms of Service and invite intellectual property litigation.

1.5.3 Recommendations for Safe Enterprise Replication

1. **Audit the Data Sources:** Strip out or mock API endpoints (like ACLED or OpenSky) until the procurement team has secured proper commercial data licenses.
2. **Lock Down the Perimeter:** Ensure the application deployment is placed behind a corporate VPN, IP allowlist, or strict Zero Trust Network Access (ZTNA) policy.
3. **Localize AI Processing:** Ensure any generative AI integrations use self-hosted models or enterprise APIs (like Azure OpenAI) to prevent proprietary trading strategies or research queries from being ingested as public training data.

Chapter 2: Security & Authentication Blueprint

2.1 Executive Summary

This chapter outlines the required security posture for the enterprise platform. Currently, the application operates as an open, read-heavy public dashboard. Transitioning this into a secure, internal enterprise tool requires patching immediate architectural vulnerabilities and implementing a robust Enterprise Identity Provider (IdP) to lock down access, protect proprietary customisations, and ensure corporate data privacy.

2.2 Security Audit & Red Flags

Based on an analysis of the core architecture, the following vulnerabilities and flaws must be addressed prior to enterprise deployment:

2.2.1 Server-Side Request Forgery (SSRF) Risks

- **The Threat:** Files acting as middleman proxies to bypass client-side CORS pose a risk. If these endpoints do not strictly validate and whitelist the URLs they fetch, malicious actors (or compromised internal accounts) could use edge functions to scan internal corporate networks.
- **Remediation:** Strict URL parsing must be implemented. A hardcoded whitelist of allowed domains (e.g., specific news RSS domains) is required, and any user-supplied URLs that do not match must be rejected.

2.2.2 Insecure Session Handling & Scraping

- **The Threat:** Local session token generation for scraping presents a massive risk of session hijacking if strings are stored locally or unencrypted on the server.
- **Remediation:** If scraping requires authenticated sessions, the session strings must be stored in an enterprise secret manager (e.g., HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault). These should never be written to local files.

2.2.3 Weak API Gateway Protection & Network Perimeter

- **The Threat:** Rudimentary key-based protection (simple string-matching checks) is entirely insufficient for an application housing proprietary trading data.
- **Remediation:** Static API keys must be replaced by dynamically generated JSON Web Tokens (JWTs) tied to corporate identities. Furthermore, the entire application should be placed behind a corporate VPN or Zero Trust Network Access (ZTNA) provider (e.g., Cloudflare Access, Tailscale).

2.2.4 Tauri Desktop IPC Boundaries

- **The Threat:** The frontend communicates with a local backend via Inter-Process Communication (IPC). If the IPC is not strictly scoped, malicious scripts could execute arbitrary local system commands on user machines.
- **Remediation:** The allowlist in Tauri capabilities must be strictly configured to ensure the frontend can only invoke specific, sanitized Rust commands.

2.3 Tech Stack Recommendations for Enterprise Identity

Because the architecture relies heavily on a React-based frontend, Edge Functions for proxying, and a real-time database (Convex), the identity solution must integrate seamlessly while supporting corporate Single Sign-On (SSO).

2.3.1 Recommended Solution: Auth0 or Microsoft Entra ID

For an internal corporate tool, consumer-focused authentication is insufficient. An Enterprise IdP like Auth0 or Microsoft Entra ID (Azure AD) is highly recommended.

- **Why:** These providers offer robust SAML/OIDC integrations, ensuring employees can log in using existing corporate credentials. Offboarding an employee instantly revokes their access to the dashboard.
- **Features:** Role-Based Access Control (RBAC) allows different proprietary datasets to be surfaced depending on whether the user belongs to the "Traders" group or the "Leadership" group.

2.4 Authentication Implementation Steps

The following steps detail how to lock down the site and inject a secure enterprise login system.

Step 1: IdP Provisioning

1. Create a Single Page Application (SPA) in the chosen IdP Dashboard.
2. Configure the connection to the internal corporate directory (e.g., Enterprise SAML, Google Workspace, or Azure AD).
3. Retrieve the necessary environment variables (Domain, Client ID) and add them to the secure environment configuration.
4. Set up an API representation to secure the edge functions.

Step 2: Frontend Integration

The application must be wrapped in the authentication provider to enforce login before the application loads.

- Dependencies (e.g., @auth0/auth0-react) are installed, and the root render of the application is wrapped in the provider context, passing the required authorization parameters.

Step 3: Secure the UI (Absolute Lockout)

Because this is a proprietary internal tool, there should be no public landing page.

- The main routing component must force authentication. If a user is unauthenticated, the application should automatically redirect them to the corporate login portal before rendering any dashboard layouts or data panels.

Step 4: Edge API Protection

To protect custom backend routes, the employee's JWT must be cryptographically validated at the edge.

- Verification middleware (e.g., express-oauth2-jwt-bearer) must be applied to all critical endpoints to authenticate requests before fetching proprietary trading or research data.

Step 5: Backend Authorization

If Convex is retained for real-time state, it must be configured to trust the Enterprise IdP.

- The backend authentication configuration is updated with the IdP domain and application ID.
- Backend mutations and queries are wrapped to require an authenticated context, ensuring only verified employees can alter the dashboard state or save custom map configurations.

Chapter 3: Data Specialization Strategy

3.1 Executive Summary

An evaluation of the current data ingestion and caching architecture demonstrates that to pivot this platform into a specialized intelligence tool for the African continent—with a deep dive into Kenya—the system must adopt an "Enrichment and Layering" strategy.

Instead of replacing the global view, the architecture will layer high-resolution local data on top of the existing global feeds. This chapter outlines the technical modifications and the specific APIs required to achieve this dual-lens (Global Macro + Local Micro) architecture.

3.2 Backend & Pipeline Modifications

The architecture relies on edge functions fetching global datasets. To enrich this with localized data, parallel fetchers and layered rendering techniques must be implemented.

3.2.1 Spatial Resolution & Mapping (Deck.gl)

- **Layered GeoJSONs:** The current base map for global borders will be retained for the macro view. A secondary mapping layer for high-resolution African administrative borders and Kenyan Counties/Wards will be added.
- **Dynamic Rendering:** Deck.gl layers will be configured to be zoom-dependent. Global borders render at wider zoom levels. As the viewport zooms in on East Africa, the granular county boundaries smoothly fade in.
- **Viewport Default:** The initial view state is adjusted to an Afro-Eurasian perspective, providing global context while naturally drawing focus to the African continent.

3.2.2 Database & Caching Adjustments

- **Tagged Caching:** Caching logic is modified to include regional tags instead of strict partitions. Global feeds are fetched, and local fetchers append data to specific tagged lists (e.g., news:global, news:africa, news:ke).
- **Schema Updates:** Data models are updated to include specialized tables for local context. Adding optional regional and county fields allows users to filter global commodity impacts down to specific local regions.
- **Currency Support:** Protobuf schemas are updated to support parallel currency representations, allowing the interface to toggle seamlessly between major global currencies (e.g., USD) and local currencies (e.g., KES).

3.2.3 Ingestion Logic Overhaul

- **Parallel Fetching:** Global news and data fetchers are not restricted. Instead, parallel ingestion scripts are created to simultaneously fetch continental and hyper-local data streams.
- **Weighted Merging:** Edge functions merge these streams before serializing them. A higher "relevance weight" is applied to African and Kenyan intelligence so it surfaces prominently in the UI.

- **AI/ML Context:** Local ML workers classifying sentiment are prompted with dual context. System prompts must define global events in relation to the local economy (e.g., analyzing how a global shipping delay specifically impacts the port of Mombasa).

3.3 Regional API Integrations

To populate the specialized dashboard layers, the existing global APIs must be supplemented with high-fidelity regional data sources.

3.3.1 Economic & Market Intelligence

Data Source	Description	Integration Target	Type
africamonitor API	Database of macroeconomic time series for Africa, augmenting global economic data.	Economic Service	Open/Public
Central Bank of Kenya (CBK)	Official FX rates, treasury bills, and macro-economic indicators.	Economic Service	Free/Public
African Markets API	Enterprise data feeds providing deep stock indices (e.g., NSE20, NASI).	Market Service	Commercial
Kenya Open Data Portal	Granular, county-level public expenditure and revenue data.	Economic Service	Open Data

3.3.2 Geographic, Infrastructure & Environment

Data Source	Description	Integration Target	Type
RCMRD Open Data	Highly localized flood risk and satellite anomalies for East Africa.	Climate Service	Free / Open
Regional Power Outage Feeds	Scrapers or APIs mapped to track local power grid infrastructure health.	Infrastructure Service	Custom / Web
IGAD Climate Prediction	Specialized weather and climate extremes data for the Greater Horn of Africa.	Climate Service	Free / Open
Regional Traffic/Logistics APIs	Mapping applied to major transport arteries to track logistical chokepoints alongside global shipping routes.	Supply Chain Service	Commercial

3.3.3 News, Unrest & Geopolitics

Data Source	Description	Integration Target	Type
Ushahidi Platform APIs	Crowdsourced crisis mapping, unrest, and local civic monitoring.	Unrest Service	Open Source
Filtered News APIs	Fetchers targeted at specific local news domains and publications.	News Service	Commercial
ACLED (Africa Desk)	Robust dataset for political violence in Africa (requires commercial license).	Conflict Service	Commercial

Africa Information Highway	Data portal API for cross-referencing socio-economic status with geopolitical events.	Intelligence Service	Open Data

3.4 Implementation Priorities

1. **Protobuf Enrichment:** Existing schema files must be updated with optional regional fields to standardise local data without breaking global schemas.
2. **Layer GeoJSONs:** High-resolution regional map data must be acquired and integrated into the map configuration, ensuring performance is maintained via zoom-dependent rendering.
3. **Build Local Adapters:** Custom edge functions must be authored to pull from local open data portals and merge them seamlessly into the existing global feeds.

Chapter 4: UI/UX & Search Implementation Guide

4.1 Executive Summary

This chapter outlines the frontend and user experience strategy for the enterprise intelligence tool. The primary objective is to reduce cognitive load for traders and leadership through a customizable, "Dual-Lens" dashboard and an ultra-low-latency, unified search experience.

4.2 Summarized Custom Dashboard (UI/UX)

The goal is to transition away from a cluttered, raw-data view, allowing users to effortlessly pin, summarize, and monitor specific intelligence streams.

4.2.1 Design Philosophy: The "Dual-Lens" Approach

- **The Canvas:** The WebGL map remains the primary spatial canvas (Macro view).
- **The Intelligence Sidebar:** A translucent, blur-backed floating panel on the side of the screen handles detailed intelligence (Micro view).
- **Card-Based Summaries:** Data is summarized via AI into concise "Intelligence Cards" (e.g., "Nairobi Port Congestion: High - 3 day delay"). Clicking a card expands it for raw, underlying metrics.
- **Pinning System:** Users can interact with a "Pin" icon on any global or regional asset to lock that specific data stream to the top of their personal sidebar.

4.2.2 Frontend Stack Recommendations

To achieve a modern, premium feel suitable for enterprise leadership:

- **CSS Framework:** Tailwind CSS is recommended for rapid UI iteration, consistent design tokens, and built-in support for dark mode and glassmorphism aesthetics.
- **UI Components:** Unstyled, accessible primitives (such as Radix UI) are recommended to build branded accordions, dialogs, and sliders.
- **State Management:** Lightweight state management libraries (such as Zustand or Nanostores) should be utilized to persist user dashboard layouts and pinned items without heavy boilerplate.

4.2.3 Dashboard Architecture

The intelligence sidebar is rendered as a reactive component. It subscribes to a user preferences store. Whenever a user pins a new regional data stream or an AI summary updates, the component dynamically re-renders the summarized cards within the sidebar container.

4.3 Advanced Search Implementation

For enterprise environments, querying a specific market event, stock ticker, or policy change must yield instantaneous results. Standard database queries are often too slow and inflexible for this requirement.

4.3.1 Search Engine Recommendations

A dual-engine approach is recommended to handle complex enterprise queries:

1. **The Keyword Engine (e.g., Meilisearch):**
    - *Purpose:* Optimized for <50ms response times and robust typo tolerance.
    - *Use Case:* Searching for specific named entities, such as ticker symbols or local infrastructure hubs.
2. **The Semantic / Vector Engine (e.g., Qdrant or Pinecone):**
    - *Purpose:* Allows natural language querying over AI-generated summaries and proprietary research databases.
    - *Use Case:* Complex contextual queries like "recent geopolitical events affecting East African supply chains."

4.3.2 The Unified Command Palette

The standard interface for modern enterprise search is the Command Palette. Instead of a traditional search bar, users trigger a centralized overlay via keyboard shortcuts (e.g., Ctrl+K).

- **Behavior:** As the user types, the UI queries the fast keyword engine. Results are instantly grouped by category (e.g., Assets, News, Economic Indicators, Geographic Hubs).
- **Integration:** When a user selects a search result, the system dispatches an event that simultaneously opens the relevant intelligence panel and triggers the map to seamlessly zoom ("fly to") the geographic coordinates associated with the event.

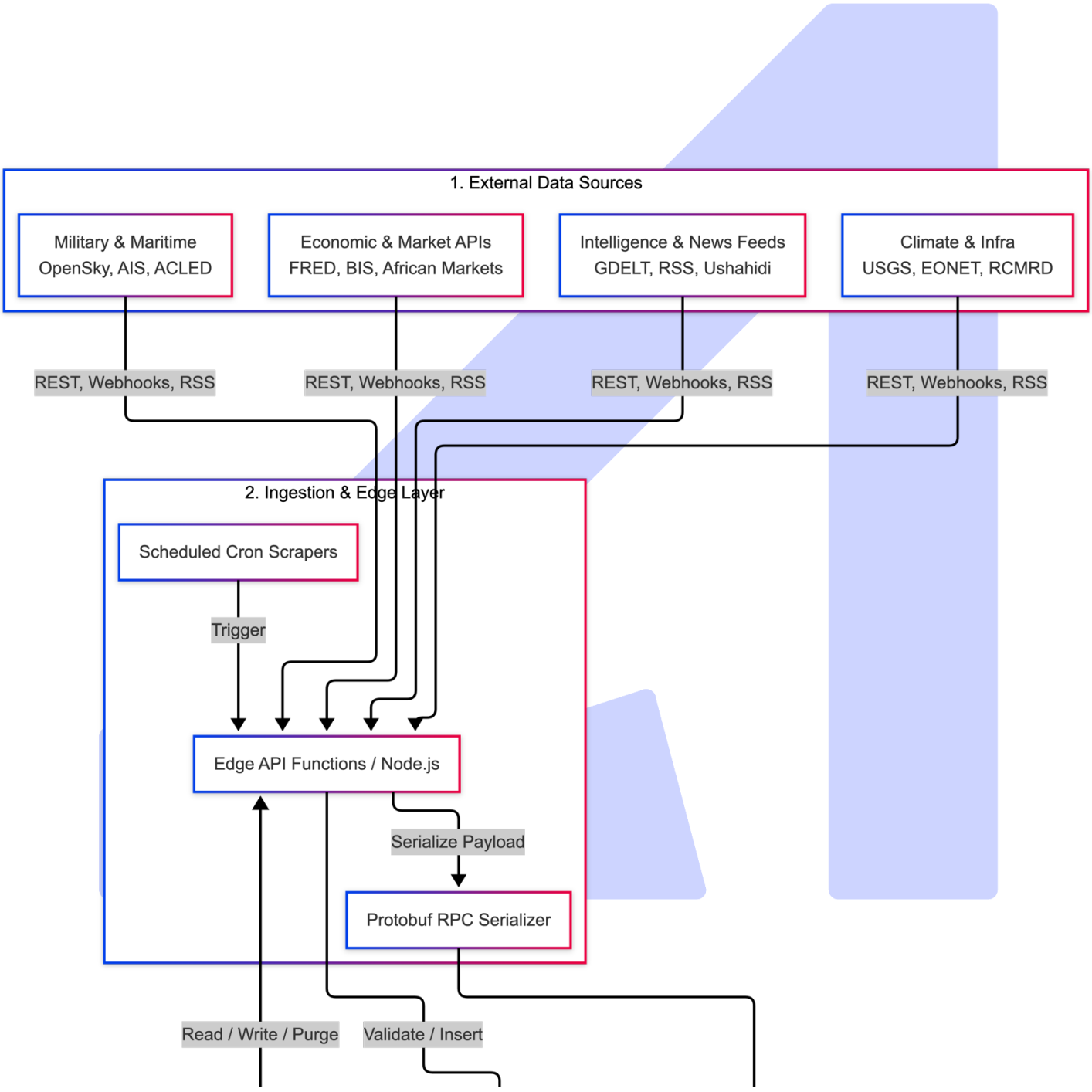
4.3.3 Data Pipeline to Search Engine

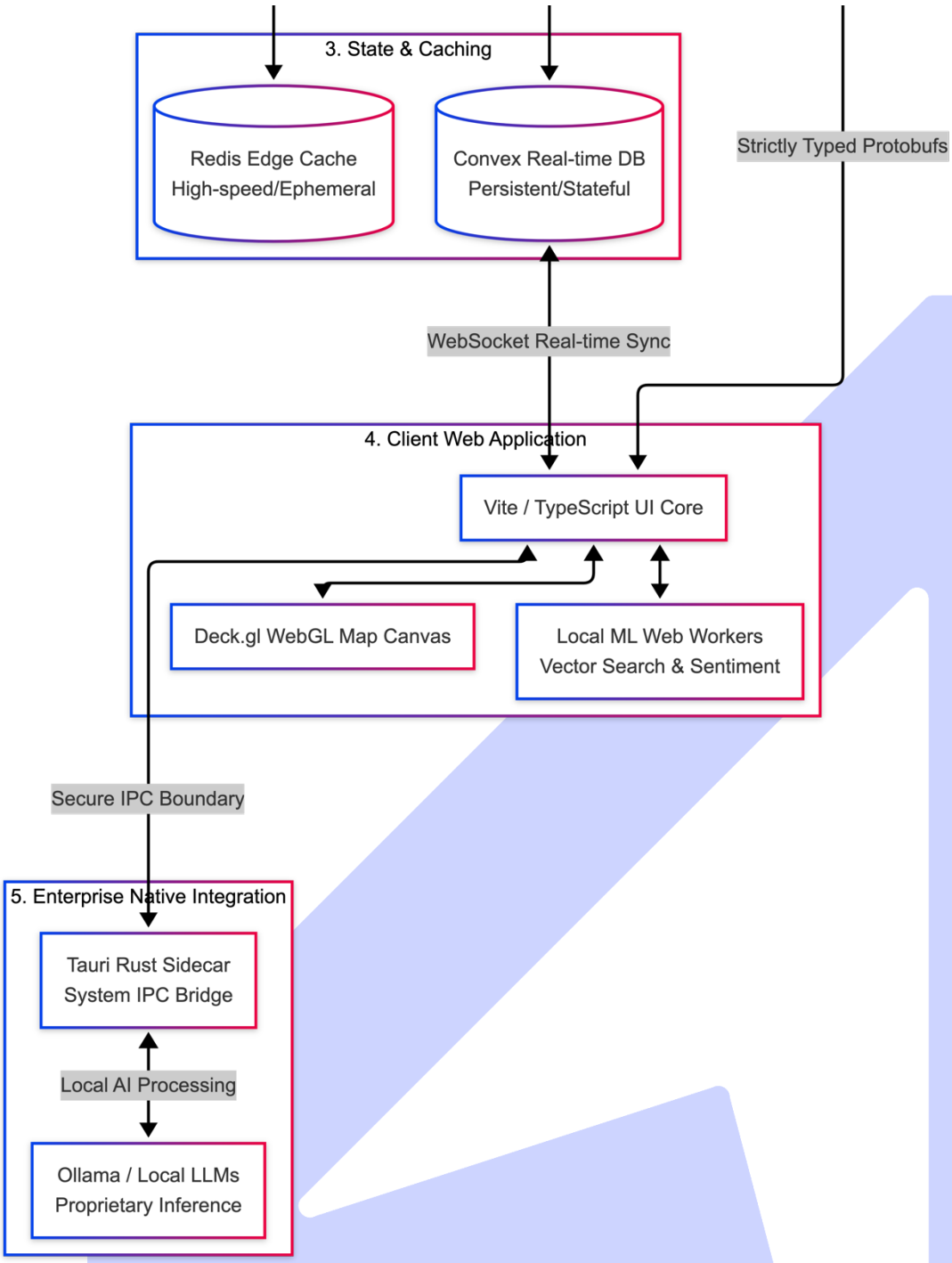
To maintain real-time search accuracy, the data ingestion edge functions must be tightly coupled with the search indexing API. Whenever a new article, price change, or regional alert is fetched and verified, the backend must immediately format a JSON payload and push it to the search index, ensuring the Command Palette always reflects the latest available intelligence.

Chapter 5: System Infrastructure Diagram

5.1 Architectural Topology

The following diagram maps the complete end-to-end data pipeline, illustrating how raw global data is ingested, processed, cached, and finally rendered on the client interfaces.




3. State & Caching
Redis Edge Cache
High-speed/Ephemeral
Convex Real-time DB
Persistent/Stateful
Strictly Typed Protobufs
WebSocket Real-time Sync
4. Client Web Application
Vite / TypeScript UI Core
Deck.gl WebGL Map Canvas
Local ML Web Workers
Vector Search & Sentiment
Secure IPC Boundary
5. Enterprise Native Integration
Tauri Rust Sidecar
System IPC Bridge
Local AI Processing
Ollama / Local LLMs
Proprietary Inference

5.2 Core Data Flow Summary

The system architecture functions as a highly decoupled, read-optimized ingestion pipeline. Raw geopolitical, economic, and environmental data is pulled from disparate external sources via serverless Edge Functions and scheduled cron scrapers. To circumvent strict rate limits from third-party vendors and guarantee sub-second latency for end users, this data is immediately routed into an aggressive Redis edge cache, while persistent user configurations are synced via the Convex real-time database. The backend normalizes the cached data and serializes it into lightweight Protocol Buffers (Protobuf) before transmitting it to the frontend application. Upon receipt, the Vite/React client hydrates the user interface, rendering spatial data onto a high-performance Deck.gl canvas, while computationally heavy tasks—such as semantic search and text summarization—are offloaded to local Web Workers and native Tauri/Ollama integrations to ensure proprietary corporate intelligence remains strictly on-device.

Chapter 6: Codebase Navigation Guide

6.1 Executive Summary

This chapter provides a comprehensive map of the WorldMonitor repository. It is designed to accelerate onboarding for engineering teams, allowing developers to rapidly locate and modify structural components, ingestion pipelines, and user interfaces as the platform is adapted for enterprise use.

6.2 Core Directory Structure

The repository follows a decoupled, monorepo-style structure separating the edge backend, the frontend client, the desktop environment, and the data schema definitions. Below is the high-level topological map of the project.

```
worldmonitor/
├── api/                    # Serverless edge functions & proxy endpoints
├── convex/                 # Real-time database schemas and mutation logic
├── data/                   # Local static JSON data sets and fallbacks
├── docs/                   # Project documentation and architectural records
├── proto/                  # Protocol Buffer definitions for API boundaries
├── public/                 # Static web assets, GeoJSON maps, and favicons
├── scripts/                # Ingestion, seeding, and automated build scripts
├── server/                 # Core backend logic, routing, and caching tier
│   ├── _shared/            # Redis caching, rate limiting, and shared utilities
│   └── worldmonitor/       # Domain-specific backend handlers (Economic, Military,
etc.)
├── src/                    # Frontend client source code (Vite/TypeScript)
│   ├── app/                # Application state, context, and data loaders
│   ├── components/         # UI components, panels, and Deck.gl map integrations
│   ├── config/             # Frontend configurations (panels, variants, geo-bounds)
│   ├── generated/          # Auto-generated Protobuf clients and servers
│   ├── locales/            # Internationalization (i18n) translation files
│   ├── services/           # Frontend service layers and API integrations
│   ├── styles/             # CSS stylesheets and global design tokens
│   ├── utils/              # Helper functions and frontend utilities
│   └── workers/            # Local Web Workers for ML and vector search
├── src-tauri/              # Tauri Rust backend for the native desktop application
└── tests/                  # Unit and integration test suites
```

6.3 Where to Find What

To streamline the rebuilding and modification process, development efforts should be directed to the following specific paths based on the architectural domain.

6.3.1 UI Components and Layouts

Modifications to the visual interface, map rendering, and user experience occur strictly within the client source.

- **UI Panels & Widgets:** **`/src/components/`** contains all individual UI elements, such as the `EconomicPanel.ts`, `SearchModal.ts`, and the primary `DeckGLMap.ts` renderer.
- **Global Styling:** **`/src/styles/`** houses the CSS frameworks, including `main.css` and `base-layer.css`, which control the glassmorphism and dark mode aesthetics.

- **Application Shell:** **`/src/app/panel-layout.ts`** and **`/src/App.ts`** dictate how the components are arranged on the screen.

6.3.2 API Fetchers and Serverless Functions

Backend modifications, data ingestion, and proxy logic are split between the edge deployment layer and internal server logic.

- **Edge Endpoints:** **`/api/`** contains the Vercel-compatible edge functions. Files like `rss-proxy.js` and `telegram-feed.js` live here and act as the immediate boundary for incoming client requests.
- **Backend Business Logic:** **`/server/worldmonitor/`** is categorized by intelligence domain (e.g., `/economic/`, `/conflict/`). Developers will find the exact TypeScript handlers that process data before it is sent to the client here.
- **Scheduled Ingestion:** **`/scripts/`** contains the Node.js scripts (e.g., `fetch-gpsjam.mjs`) responsible for periodically scraping and fetching external data sources.

6.3.3 State Management and Caching Logic

The platform relies on aggressive caching to maintain low latency and manage state across the application.

- **Backend Caching (Redis):** **`/server/_shared/redis.ts`** and **`/server/_shared/cache-keys.ts`** control the server-side caching lifecycle, TTLs, and cache invalidation logic.
- **Frontend State:** **`/src/app/`** contains files like `app-context.ts` and `data-loader.ts` which manage the in-memory state of the active viewport and user selections.
- **Real-Time Data Sync:** **`/convex/registerInterest.ts`** handles real-time subscriptions for active data streams.

6.3.4 Configurations (Environment, Routing, Themes)

Application-wide settings spanning both build-time and runtime configurations.

- **Environment & Deployment:** The root directory contains **`.env.example`**, **`vercel.json`** (edge routing and headers), and **`vite.config.ts`** (frontend build tooling).
- **Frontend Configurations:** **`/src/config/`** dictates how the application behaves. Files here define the base geographic coordinates (`geo.ts`), entity definitions (`entities.ts`), and application variants (`variants/`).
- **Backend Routing:** **`/server/router.ts`** maps incoming RPC requests to their respective internal domain handlers.
- **Middleware:** **`/middleware.ts`** (in the root) handles request interception, which is the exact location to inject global authentication verification.

6.3.5 Data Definitions and Schemas

Data structures are strictly typed to ensure consistency across the backend and frontend boundary.

- **API Schemas (Protobuf):** **`/proto/worldmonitor/`** contains all `.proto` files detailing the exact shape of the data payloads (e.g., `economic_data.proto`). Modifying these requires running the `buf` generation tools to update the types.

- **Database Schemas:** `/convex/schema.ts` defines the persistent storage tables and relationships.
- **Static Geospatial Data:** `/public/data/` houses the raw shapefiles, such as `countries.geojson`, which are directly loaded by the Deck.gl map.

Chapter 7: Draft Development Guide: Architecture & UI

7.1 Executive Summary

This chapter outlines the proposed technical implementation for transforming the WorldMonitor codebase into the AIB-AXYS Africa Enterprise Intelligence Portal. It details the migration from a serverless real-time database to a robust relational architecture, the construction of a modular dashboard, and the implementation of critical security patches.

[DEVELOPER REVIEW REQUIRED: This document serves as a draft proposal. The engineering team is strongly encouraged to critically review all architectural choices, suggest alternative patterns, and identify potential bottlenecks before these standards are officially ratified.]

7.2 Authentication & Database Architecture

The existing architecture relies on Convex for state and database management. To meet enterprise requirements for data sovereignty, complex relational querying, and seamless integration with corporate Single Sign-On (SSO), the system will transition to a PostgreSQL foundation paired with NextAuth/Auth.js.

7.2.1 ORM and Database Selection

The backend will utilize PostgreSQL as the primary data store. Interaction with the database will be managed via either **Prisma** or **Drizzle ORM**.

- **Prisma** offers a highly readable schema definition and rapid prototyping capabilities.
- **Drizzle ORM** provides superior edge-compatibility and SQL-like syntax, which may be preferable if the application remains heavily reliant on Vercel Edge functions.

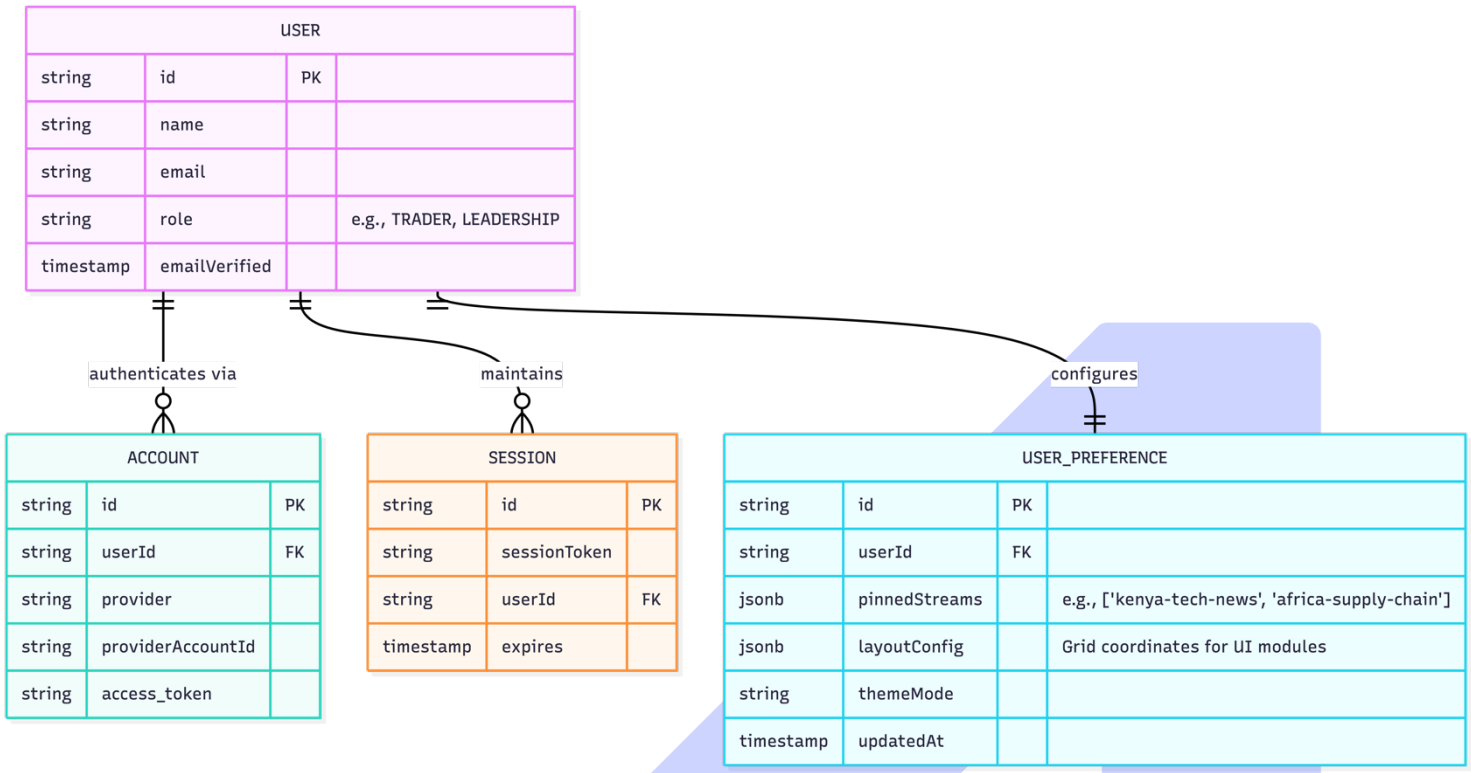
[DEVELOPER REVIEW REQUIRED: The engineering team must evaluate Prisma vs. Drizzle ORM against the current deployment pipeline (Vercel Edge vs. Node Serverless) and benchmark cold-start performance to make a final selection.]

7.2.2 Identity via NextAuth / Auth.js

Authentication will be handled by NextAuth (Auth.js), configured to support enterprise providers (e.g., Microsoft Entra ID, Okta) via SAML or OIDC. NextAuth will interact directly with the PostgreSQL database using the standard adapter pattern to manage users, accounts, and sessions.

7.2.3 Proposed Database Schema

The following Entity-Relationship Diagram maps the baseline schema required to support enterprise identity and custom dashboard states.



7.3 The "Easy-on-the-Eye" Dynamic Dashboard

To reduce cognitive load for traders and executives, the dense, data-heavy Deck.gl map interface will be relocated to a dedicated `/map` route. The default root route (`/`) will be rebuilt as a modular, highly legible intelligence dashboard.

7.3.1 UI/UX Layout Strategy

- **Modular Grid:** The dashboard will utilize a responsive CSS Grid layout.
- **Widget-Based Intelligence:** Information will be displayed in discrete, summary-focused widgets (e.g., "Nairobi Securities Exchange Tracker," "Red Sea Shipping Alerts").
- **Personalization:** Users will have the ability to pin, unpin, and drag-and-drop these widgets. A trader focused on East Africa might pin "Kenyan Tech News" and "African Supply Chain Alerts" to the top of their view, while hiding global macro indicators.

7.3.2 Frontend State Management

To handle the highly dynamic nature of a customizable widget grid, **Zustand** is proposed as the frontend state management solution.

- Zustand will maintain the local state of the `layoutConfig` and `pinnedStreams`.
- When a user modifies their dashboard, Zustand will optimistically update the UI while asynchronously dispatching a `PUT` request to the backend to update the `USER_PREFERENCE` table in PostgreSQL.

[DEVELOPER REVIEW REQUIRED: The team must assess whether Zustand provides sufficient tools for this optimistic UI pattern, or if leveraging React Context paired with a data-fetching library like React Query / SWR is more appropriate for handling server-state synchronization.]

7.4 Security Patching & Hardening

The current public-facing repository contains architectural patterns that present severe risks in an enterprise environment. The following remediation steps must be implemented.

7.4.1 Mitigating SSRF in Proxy Endpoints

Existing proxy endpoints (e.g., `api/rss-proxy.js`, `api/telegram-feed.js`) currently accept arbitrary URLs. This Server-Side Request Forgery (SSRF) vulnerability allows actors to force the server to make requests to internal corporate IP addresses.

- **Implementation Step:** Developers must replace arbitrary URL acceptance with strict, hardcoded whitelists.
- **Validation Pattern:** The backend must use the `URL` constructor to parse incoming requests, ensuring the `hostname` exactly matches an approved list (e.g., `['nation.africa', 'cbk.go.ke']`) before initiating the fetch.

7.4.2 Securing API Gateways

Static API keys must be completely removed from the codebase.

- **Implementation Step:** All endpoints under `/api/` and `/server/` must be wrapped in authentication middleware.
- **Validation Pattern:** Using Auth.js, the middleware must extract the session token from the request headers/cookies, verify its cryptographic signature against the database, and reject any unauthorized requests with a `401 Unauthorized` status.

7.4.3 Robust Rate Limiting

To prevent abuse and ensure high availability for critical trading data, the application must enforce strict rate limits on all data-fetching routes.

- **Implementation Step:** Integrate an Upstash Redis (or equivalent Redis cluster) rate-limiting module.
- **Validation Pattern:** Implement a sliding window algorithm. API requests should be limited per `userId` (extracted from the authenticated session) rather than by IP address, ensuring fair usage among internal employees while preventing scraper abuse.

[DEVELOPER REVIEW REQUIRED: The security team must conduct a full audit of all edge functions to ensure no proxy endpoint is overlooked during the SSRF whitelisting phase. Feedback is requested on the threshold values for the Redis rate-limiting algorithms.]

Chapter 8: Draft Deployment Guide: Bulletproof CI/CD

8.1 Executive Summary

This chapter outlines a robust, zero-downtime deployment strategy for the enterprise intelligence portal. The previous methodology (manual SFTP and PID killing) is insufficiently resilient for a production-grade application. This guide details a modernized PM2 ecosystem approach, ensuring high availability, streamlined updates, and rapid rollback capabilities.

[DEVELOPER REVIEW REQUIRED: This procedure must be rigorously tested in a staging environment by the engineering team before being applied to the production server.]

8.2 Modern Infrastructure Proposal

To achieve zero-downtime deployments on a dedicated Linux server, the architecture will utilize **PM2 in Cluster Mode** placed behind an **Nginx Reverse Proxy**.

8.2.1 Why PM2 over Docker?

While Docker Compose is excellent for containerization, a bare-metal PM2 approach is often more straightforward for Next.js/Vite applications deployed to single, high-performance instances. PM2's native `reload` command leverages Node.js cluster capabilities to spawn new worker processes and seamlessly traffic-shape incoming requests before killing old workers, resulting in true zero-downtime deployments without the overhead of container orchestration.

8.2.2 Architecture Overview

1. **Nginx:** Acts as the public-facing web server, handling SSL termination (via Let's Encrypt) and proxying traffic to the internal PM2 processes.
2. **PM2 Ecosystem:** Manages the Node.js application instances, ensuring they remain online, restarting them upon failure, and distributing load across CPU cores.
3. **CI/CD Script:** A bash script automates the sequence of pulling code, installing dependencies, building the application, and triggering the PM2 reload.

8.3 Step-by-Step Server Provisioning

The following instructions detail the initial setup required on a fresh Linux server (e.g., Ubuntu 22.04 LTS).

Step 1: System Updates and Dependencies

Connect via SSH and update the system packages.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git build-essential nginx
```

Step 2: Install Node.js and PM2

Install a stable Node.js version (e.g., v20 LTS) using NVM (Node Version Manager) to prevent permission issues.

```
# Install NVM
curl -o- [https://raw.githubusercontent.com/nvm-
sh/nvm/v0.39.7/install.sh](https://raw.githubusercontent.com/nvm-
sh/nvm/v0.39.7/install.sh) | bash
source ~/.bashrc

# Install Node and PM2
nvm install 20
nvm use 20
nvm alias default 20
npm install -g pm2 yarn pnpm
```

Step 3: Clone the Repository

Generate an SSH key on the server (`ssh-keygen -t ed25519`), add it to the corporate GitHub/GitLab account as a deployment key, and clone the repository.

```
mkdir -p /var/www/worldmonitor
cd /var/www
git clone git@github.com:your-org/worldmonitor.git
cd worldmonitor
```

Step 4: Configure the PM2 Ecosystem

Create an `ecosystem.config.js` file in the root of the repository to define how PM2 manages the application.

```
module.exports = {
  apps: [{
    name: 'worldmonitor-enterprise',
    script: 'npm',
    args: 'start', // Assuming standard Vite/Next.js start command
    instances: 'max', // Utilize all available CPU cores
    exec_mode: 'cluster', // Enables zero-downtime reloads
    env: {
      NODE_ENV: 'production',
      PORT: 3000
    }
  }]
}
```

Start the application for the first time:

```
pm2 start ecosystem.config.js
pm2 save
pm2 startup # Follow the on-screen instructions to enable startup on boot
```

Step 5: Configure Nginx Reverse Proxy

Create a new Nginx configuration file.

```
sudo nano /etc/nginx/sites-available/worldmonitor
```

Add the following configuration, ensuring the proxy passes traffic to the PM2 port (default 3000):

```
server {
    listen 80;
    server_name portal.yourcompany.com;

    location / {
        proxy_pass [http://127.0.0.1:3000](http://127.0.0.1:3000);
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Enable the site and restart Nginx.

```
sudo ln -s /etc/nginx/sites-available/worldmonitor /etc/nginx/sites-
enabled/
sudo nginx -t
sudo systemctl restart nginx
```

(Note: Use `certbot` to secure the domain with SSL/TLS immediately after this step).

8.4 The Automation Script (`deploy.sh`)

To eliminate fragile manual deployments, place the following `deploy.sh` script in the server's application directory (or execute it via a GitHub Action over SSH).

This script is designed to fail gracefully. If the build step fails, the PM2 reload is aborted, ensuring the application remains online serving the previous stable version.

```
#!/bin/bash#
================================================================================
# WorldMonitor Enterprise - Zero-Downtime Deployment Script#
================================================================================

set -e # Exit immediately if any command exits with a non-zero status
echo "[START] Starting deployment for WorldMonitor Enterprise..."

APP_DIR="/var/www/worldmonitor"
BRANCH="main"

cd $APP_DIR

echo "[PULL] Pulling latest changes from $BRANCH..."
git fetch origin
git reset --hard origin/$BRANCH

echo "[INSTALL] Installing dependencies..."
npm ci # Use npm ci for clean, deterministic installs based on package-
lock.json

echo "[BUILD] Building the application..."
# If the build fails, the script exits here due to 'set -e', preventing
downtime
npm run build
```

```
# Example: Run database migrations if applicable (e.g., Prisma)
# echo "[MIGRATE] Running database migrations..."
# npx prisma migrate deploy

echo "[RELOAD] Reloading PM2 instances..."
# pm2 reload initiates a zero-downtime cluster restart
pm2 reload worldmonitor-enterprise --update-env

echo "[SUCCESS] Deployment completed successfully!"
```

8.4.1 Rollback Procedure

If an issue is detected post-deployment, the process can be quickly reverted:

1. SSH into the server and navigate to the application directory.
2. Find the previous stable commit hash (`git log`).
3. Reset the repository to that commit (`git reset --hard <commit_hash>`).
4. Re-run the deployment script (`./deploy.sh`).

Chapter 9: AI Execution Prompts for Developers

9.1 Executive Summary

This chapter provides a structured AI Execution Manual. It contains highly specific, stage-by-stage prompts designed to be copied and pasted directly into AI CLI tools (such as Claude Code) or AI-integrated IDEs (such as Cursor). These prompts will execute the architectural transformations outlined in Chapter 7.

[CRITICAL DIRECTIVE: Human oversight is mandatory. Developers must rigorously review, test, and validate the AI's output at the conclusion of every single prompt. Do not blindly commit generated code without verifying it against the overarching enterprise security and architectural standards.]

9.2 Phase 1: Database and Authentication Initialization

The first phase strips out the legacy serverless database (Convex) and initializes the enterprise-grade PostgreSQL and Auth.js foundation.

Step 1.1: Remove Convex and Prepare Dependencies Copy the following prompt into the AI agent:

```
Analyze the current repository. Identify and remove all files, folders, and
dependencies related to 'convex'. This includes the 'convex/' directory,
'convex' in package.json, and any ConvexProvider wrappers in the frontend
initialization files (e.g., src/main.ts or src/App.ts). Once removed,
install the following dependencies for our new architecture: Prisma (and
Prisma Client), Auth.js (NextAuth core packages suitable for our Edge/Node
environment), and the Prisma adapter for Auth.js. Provide a brief summary
of the files modified and deleted.
```

Step 1.2: Initialize Prisma and Auth.js Base Copy the following prompt into the AI agent:

```
Initialize a new Prisma project targeting a PostgreSQL database. Generate
the '.env' variables needed for the database connection and the Auth.js
secret. Next, create the foundational Auth.js configuration file (e.g.,
auth.config.ts) configured to use the Prisma adapter and placeholder
enterprise SSO providers (like Microsoft Entra ID or an OIDC provider). Do
not configure the specific UI yet, just the backend initialization. Pause
for review once the Prisma schema file is created and the Auth
configuration is scaffolded.
```

9.3 Phase 2: Database Schema Generation

This phase translates the enterprise data model into a functional database schema capable of storing custom dashboard preferences.

Step 2.1: Construct the Relational Schema Copy the following prompt into the AI agent:

```
Update the 'schema.prisma' file to reflect the following Entity-
Relationship logic for our enterprise portal.
```

```
1. Standard Auth.js models: User, Account, and Session. The User model must
include a 'role' string field (defaulting to 'TRADER').

2. A 'UserPreference' model that has a one-to-one relationship with the
User.

3. The UserPreference model must include the following fields: 'id'
(String/UUID), 'userId' (relation to User), 'pinnedStreams' (JSON),
'layoutConfig' (JSON), 'themeMode' (String), and 'updatedAt' (DateTime).

After writing the schema, generate the migration command needed to push
this to the PostgreSQL database, but do not execute it yet.
```

9.4 Phase 3: Dynamic Dashboard Construction

This phase reorganizes the routing and builds the "easy-on-the-eye" modular dashboard using modern frontend state management.

Step 3.1: Route Relocation and Zustand Setup Copy the following prompt into the AI agent:

```
First, relocate the existing complex Deck.gl map interface (currently the
default view) to a dedicated component and set up basic client-side routing
so the map is accessible only at the '/map' path. Next, install 'zustand'
and create a global store file (e.g., src/store/useDashboardStore.ts). This
Zustand store must manage 'layoutConfig' (an array of grid coordinates) and
'pinnedStreams' (an array of string IDs). Include asynchronous actions in
the store to optimistically update the UI while sending a PUT request to
sync the state with the backend UserPreference database table.
```

Step 3.2: Modular Dashboard Layout Copy the following prompt into the AI agent:

```
Create a new 'HomeDashboard' component to serve as the root route ('/').
Use Tailwind CSS to build a highly legible, responsive CSS Grid layout. The
grid should dynamically render 'Widget' components based on the
'layoutConfig' and 'pinnedStreams' stored in the Zustand store. Create a
generic 'IntelligenceWidget' component with a clean, glassmorphic design
(using Tailwind's backdrop-blur and border utilities) that can accept a
'streamId' prop and display placeholder summary text. Ensure the layout
supports drag-and-drop or simple pinning actions to update the Zustand
store.
```

9.5 Phase 4: Security Hardening

The final phase implements the mandatory security patches to prevent SSRF, secure the API, and mitigate scraping abuse.

Step 4.1: SSRF Mitigation in Proxies Copy the following prompt into the AI agent:

```
Audit all edge functions and proxy routes in the 'api/' and 'server/'
directories (specifically looking at rss-proxy.js, telegram-feed.js, and
youtube/embed.js). Refactor these endpoints to eliminate Server-Side
Request Forgery (SSRF) vulnerabilities. Implement a strict, hardcoded
whitelist array of allowed domains. Use the native URL constructor to parse
the incoming target URL, verify its 'hostname' against the whitelist, and
```

immediately return a 403 Forbidden status if the domain is not explicitly approved.

Step 4.2: API Gateway Protection Copy the following prompt into the AI agent:

```
Remove any static API key validation logic found in files like 'api/_api-key.js'. Replace this by implementing authentication middleware across all data-fetching endpoints in the '/api/' and '/server/' directories. Use the Auth.js session validation methods to extract the session token from the request cookies or headers. If no valid enterprise session exists, reject the request with a 401 Unauthorized status.
```

Step 4.3: Robust Rate Limiting Copy the following prompt into the AI agent:

```
Install the '@upstash/redis' package (or similar Redis client). Update the 'server/_shared/rate-limit.ts' file to implement a sliding window rate-limiting algorithm. Critically, the rate limiter must use the authenticated 'userId' (extracted from the Auth.js session) as the identifier key, rather than the IP address. Configure reasonable default thresholds for an internal corporate tool (e.g., 100 requests per minute per user) and ensure the endpoints return a 429 Too Many Requests status when the limit is exceeded.
```

[FINAL REVIEW DIRECTIVE: Following the execution of Phase 4, the engineering team must run the complete test suite and manually verify the application within a staged environment to ensure zero regression in data ingestion performance and absolute enforcement of authentication parameters.]

Chapter 10: Data Pipeline & ETL Strategy

10.1 Executive Summary

This chapter outlines the Data Ingestion and Extract, Transform, Load (ETL) strategy required to support the migration from Convex to a PostgreSQL-backed architecture. To maintain the continuous, real-time fetching of African and Kenyan intelligence streams without congesting the main application thread, a resilient background worker system must be established. Additionally, an orchestrated caching strategy is defined to guarantee instantaneous load times for the "easy-on-the-eye" enterprise dashboard.

10.2 Background Worker Architecture

The legacy architecture utilized Convex's internal scheduling and background functions. In a modernized Next.js/Node edge environment, standard tools like `node-cron` or `BullMQ` are often suboptimal due to their reliance on long-running processes and persistent connections, which conflict with serverless paradigms.

10.2.1 Recommended Orchestrator: Inngest (or Trigger.dev)

To replace Convex's background functions, a serverless-native event orchestration engine like **Inngest** (or alternatively, Trigger.dev) is strongly recommended.

- **Why Serverless Orchestration:** These platforms execute background jobs via HTTP webhooks, allowing data pipelines to run seamlessly on Vercel or AWS Lambda without requiring a dedicated continuous server.
- **Built-in Resilience:** They provide native features essential for third-party API ingestion, including automatic retries with exponential backoff, rate-limit queueing, and step-function execution. If the Central Bank of Kenya API experiences an outage, the orchestrator will automatically pause and retry the ingestion job without dropping data.
- **Event-Driven Execution:** Data ingestion can be triggered via scheduled cron expressions (e.g., fetching Kenyan stock indices every 5 minutes during market hours) or via discrete application events.

10.3 Multi-Tier Caching & Database Strategy

To ensure the modular UI widgets load instantaneously, the system must never query external APIs or perform heavy PostgreSQL aggregates synchronously during a user request. The architecture demands a strict separation between ingestion, storage, and presentation.

10.3.1 The Ingestion Flow (Write Path)

1. **Fetch & Transform:** The background orchestrator (Inngest) triggers a job to fetch data from regional APIs (e.g., Ushahidi, African Markets). The worker transforms this raw JSON into a normalized schema.
2. **Persistent Storage:** The normalized data is written to the **PostgreSQL** database via Prisma. This serves as the absolute Source of Truth and historical archive.
3. **Cache Invalidation & Hydration:** Immediately upon a successful database write, the worker computes the UI-ready data (e.g., calculating the 24-hour percentage change) and pushes this pre-computed payload into **Redis**.

10.3.2 The Presentation Flow (Read Path)

- **Tier 1: Redis (The UI Cache):** When the frontend dashboard requests data for a specific widget (e.g., "Nairobi News Summary"), the API route fetches directly from Redis. Redis serves this request in sub-10 milliseconds.
- **Tier 2: Stale-While-Revalidate:** If the Redis key is missing or expired, the system instantly returns a slightly stale version (if available) while asynchronously querying PostgreSQL to rebuild the cache, ensuring the user is never blocked by a database cold start.

10.4 AI Execution Prompts for Developers

The following prompts are designed to be fed into an AI CLI tool (such as Claude Code or Cursor) to rapidly scaffold the new data pipeline architecture.

Step 1: Initialize Inngest and Basic Configuration Copy the following prompt into the AI agent:

```
Install the 'inngest' SDK. Create a new directory at 'src/inngest/' and
scaffold the base configuration file ('client.ts') initializing the Inngest
client. Next, set up the Next.js/Node API route handler (e.g.,
'app/api/inngest/route.ts' or 'api/inngest.ts' depending on our routing
structure) that will serve as the webhook endpoint for Inngest to trigger
our background jobs. Provide a summary of the configuration.
```

Step 2: Create a Scheduled ETL Job for African Markets Copy the following prompt into the AI agent:

```
Using the Inngest client, create a new scheduled function (cron) that runs
every 15 minutes during weekday business hours. This function must act as
an ETL pipeline.

Step 1: Fetch mock data representing the Nairobi Securities Exchange
(NSE20) index.

Step 2: Use Prisma to save these new index values into our PostgreSQL
'MarketData' table.

Step 3: Update our Redis cache (using the existing Redis utility in
'server/_shared/redis.ts') with the latest pre-calculated summary widget
payload under the key 'widget:nse20:latest'.

Ensure the Inngest function uses step.run() blocks for each distinct
operation to guarantee automatic retries on failure.
```

Step 3: Build the Ultra-Fast Widget API Route Copy the following prompt into the AI agent:

```
Create a fast API endpoint for the frontend dashboard widgets to consume
(e.g., 'api/widgets/nse20'). This endpoint must strictly adhere to our
multi-tier caching strategy. First, attempt to retrieve the data from Redis
using the key 'widget:nse20:latest'. If the data exists, return it
immediately with a Cache-Control header enabling stale-while-revalidate.
Only if the Redis cache is completely empty should the endpoint fallback to
querying the Prisma PostgreSQL database, rebuilding the Redis cache, and
returning the response. Include thorough error handling.
```

CRITICAL DIRECTIVE: Human oversight is mandatory. Developers must rigorously review the generated ETL functions to ensure they properly handle regional API rate limits and do not leak database connection limits during parallel executions.

Chapter 11: QA & Testing Playbook

11.1 Executive Summary

This chapter outlines the comprehensive Quality Assurance (QA) and testing strategy for the enterprise intelligence portal. With the introduction of a PostgreSQL database, NextAuth identity management, and a dynamic frontend dashboard, the testing paradigm must evolve to validate secure data access and user preference persistence. This playbook details the required Unit and End-to-End (E2E) tests, specific Playwright integrations for mocking authenticated states in CI/CD pipelines, and AI execution prompts for rapid test generation.

[DEVELOPER REVIEW REQUIRED: The QA and engineering teams must review the proposed mocking strategies to ensure they align with the specific NextAuth configuration and deployment pipeline constraints before integrating into the main CI/CD workflow.]

11.2 Core Testing Strategy

To ensure absolute reliability and security of the proprietary platform, testing is divided into discrete layers focusing on identity, database integrity, and UI state.

11.2.1 Unit Testing Layer (Vitest / Jest)

- **Authentication Adapters:** Validate that custom NextAuth callbacks correctly assign the `TRADER` or `LEADERSHIP` roles based on the SSO provider's payload. The database adapter must be mocked to prevent actual PostgreSQL inserts during unit test execution.
- **State Management:** Test the Zustand (or React Context) store responsible for the dashboard layout. Ensure actions like `pinStream`, `unpinStream`, and `updateLayout` correctly mutate the local state and trigger the debounced network synchronization functions.
- **Data Transformation:** Validate the utility functions that format raw external API data into the standardized widget payloads.

11.2.2 End-to-End Testing Layer (Playwright)

- **Secure Login Flow:** Validate the redirection sequence. An unauthenticated user attempting to access `/` must be intercepted and routed to the SSO login portal.
- **Dashboard Customization:** Simulate a user dragging, dropping, and pinning an intelligence widget (e.g., "Kenyan Tech News"). The test must verify that the UI updates immediately and that the corresponding HTTP PUT request is dispatched to the backend.
- **Persistence Validation:** Simulate a page reload after a dashboard layout change to verify that the application successfully fetches and applies the saved `layoutConfig` from the backend.

11.3 Playwright Integration: Mocking Authentication

Running E2E tests in a CI/CD environment (like GitHub Actions) against a live PostgreSQL database is slow, fragile, and poses security risks. Playwright must be configured to mock the authenticated state and backend API responses.

11.3.1 Network Interception Strategy

NextAuth relies on the `/api/auth/session` endpoint to verify the user's active session on the client side. By intercepting this network request, Playwright can trick the application into behaving as if a secure enterprise user is fully logged in.

1. **Global Session Mocking:** Use Playwright's `page.route()` to intercept any requests to `/api/auth/session` and return a static, mocked JSON session object containing a mock `userId` and `role`.
2. **API Response Mocking:** Intercept requests to the user preference endpoints (e.g., `GET /api/preferences`) and return a mock `layoutConfig`. This completely bypasses the need for the PostgreSQL database during UI tests.

11.3.2 Example Playwright Setup Pattern

Instead of authenticating via the UI for every test, a `beforeEach` hook is utilized to inject the mock state:

```
// Conceptual Playwright Hook
test.beforeEach(async ({ page }) => {
  // Mock the NextAuth session
  await page.route('**/api/auth/session', route => {
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({
        user: { name: 'Test Trader', email: 'trader@company.com', role:
'TRADER' },
        expires: '2099-01-01T00:00:00.000Z',
      }),
    });
  });

  // Mock the database response for dashboard preferences
  await page.route('**/api/user/preferences', route => {
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({ pinnedStreams: ['kenya-tech'], layoutConfig:
[] }),
    });
  });
});
```

11.4 AI Execution Prompts for Developers

The following prompts are designed to be copied directly into an AI CLI tool to automatically generate the Playwright test suites based on the mocking strategy defined above.

Step 1: Generate Playwright Authentication Fixtures Copy the following prompt into the AI agent:

```
Create a Playwright test fixture file (e.g., 'tests/fixtures/auth.ts') that
extends the base Playwright test object. This fixture should automatically
intercept requests to '**/api/auth/session' and return a valid mocked
NextAuth session object for an enterprise user with the role 'TRADER'.
Also, include a secondary fixture variant for a user with the role
```

'LEADERSHIP'. Provide the complete TypeScript code for these fixtures so they can be imported into our E2E test files.

Step 2: Generate E2E Dashboard Customization Test Copy the following prompt into the AI agent:

Write a Playwright E2E test suite (e.g., 'tests/e2e/dashboard-preferences.spec.ts') that imports the 'TRADER' auth fixture created previously. The test should navigate to the root dashboard route ('/').

Step 1: Mock the 'GET /api/user/preferences' endpoint to return an empty dashboard.

Step 2: Mock the 'PUT /api/user/preferences' endpoint to intercept the save request and return a 200 OK status.

Step 3: Write the Playwright commands to simulate a user clicking a "Pin Widget" button on the UI.

Step 4: Assert that the UI successfully reflects the pinned widget, and assert that the 'PUT /api/user/preferences' network request was fired with the correct JSON payload.

Step 3: Generate Zustand Unit Tests Copy the following prompt into the AI agent:

Write a comprehensive unit test suite using Vitest (or Jest) for the Zustand dashboard store ('src/store/useDashboardStore.ts'). The tests must validate the default initial state, verify that the 'pinStream' action correctly adds a stream ID to the array without duplicates, and verify that the 'unpinStream' action correctly removes it. Include a test that mocks the global 'fetch' API to ensure the asynchronous state synchronization function handles network failures gracefully and reverts the optimistic UI update.

[FINAL REVIEW DIRECTIVE: The QA team must execute the generated Playwright scripts locally using the `--debug` flag to visually confirm that network requests are being correctly intercepted and that no live database connections are attempted during the test run.]

Chapter 12: Monitoring & Disaster Recovery Guide

12.1 Executive Summary

This chapter establishes the Day-2 operations framework for the enterprise intelligence portal. Ensuring high availability, data integrity, and rapid recovery are paramount for systems supporting trading and leadership decisions. This guide outlines a lightweight, privacy-respecting monitoring stack, an automated database backup strategy, and a definitive three-step rollback protocol to execute in the event of a critical failure.

12.2 Application Monitoring Strategy

Heavyweight Application Performance Monitoring (APM) tools can introduce unnecessary overhead and complexity. For this Linux-based deployment, a streamlined, privacy-respecting stack is implemented to ensure external deployment developers can quickly diagnose system health.

12.2.1 Local Logging: Pino

The Node.js and Next.js backend services must utilize **Pino** as the primary logger.

- **Rationale:** Pino is a low-overhead, highly performant JSON logger. It structuring logs uniformly, making them easy to parse without bogging down the event loop.
- **Privacy:** Developers must configure Pino to redact sensitive fields (e.g., session tokens, passwords, and proprietary API keys) before they are written to disk.

12.2.2 Server-Level Observability: PM2

Since the application runs via PM2, native PM2 monitoring tools provide immediate, zero-configuration insights.

- **Live Dashboards:** Developers can execute `pm2 monit` on the server to view real-time CPU, memory usage, and basic request metrics across all cluster instances.
- **Log Aggregation:** The command `pm2 logs` aggregates standard output and standard error from the Pino loggers across all worker threads.

12.2.3 Error Tracking: Sentry

For application-level exception tracking, **Sentry** is integrated into the frontend and edge functions.

- **Configuration:** Sentry must be configured with strict privacy controls. Data scrubbing rules must be enforced in the `sentry.client.config.ts` and `sentry.server.config.ts` files to ensure Personally Identifiable Information (PII) and proprietary financial queries never leave the corporate network.

12.3 Database Backup Protocol

To protect the PostgreSQL database containing user preferences, system configurations, and historical market data, daily encrypted backups must be automated.

12.3.1 Automated Backup Script

The following bash script (`backup_db.sh`) performs a full database dump, compresses it, and encrypts the output using a symmetric AES-256 cipher.

Create this script at `/opt/scripts/backup_db.sh` and grant it execution permissions (`chmod +x /opt/scripts/backup_db.sh`).

```
#!/bin/bash
#
================================================================================
# Automated PostgreSQL Backup and Encryption
#
================================================================================

set -e

# Configuration
DB_NAME="worldmonitor_enterprise"
DB_USER="postgres"
BACKUP_DIR="/var/backups/postgresql"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
RAW_FILE="$BACKUP_DIR/$DB_NAME_$DATE.sql.gz"
ENCRYPTED_FILE="$BACKUP_DIR/$DB_NAME_$DATE.sql.gz.gpg"

# Encryption Passphrase (Must be injected via secure environment variable
or vault)
PASSPHRASE=$DB_BACKUP_PASSPHRASE

mkdir -p "$BACKUP_DIR"

echo "[INFO] Starting database backup for $DB_NAME..."

# Step 1: Dump and compress the database
pg_dump -U "$DB_USER" "$DB_NAME" | gzip > "$RAW_FILE"

# Step 2: Encrypt the backup
echo "[INFO] Encrypting backup file..."
gpg --symmetric --cipher-algo AES256 --batch --passphrase "$PASSPHRASE" -o
"$ENCRYPTED_FILE" "$RAW_FILE"

# Step 3: Clean up the unencrypted raw file
rm "$RAW_FILE"

# Step 4: Retention Policy - Delete backups older than 14 days
echo "[INFO] Cleaning up backups older than 14 days..."
find "$BACKUP_DIR" -type f -name "*.gpg" -mtime +14 -exec rm {} \;

echo "[SUCCESS] Backup completed: $ENCRYPTED_FILE"
```

12.3.2 Cron Job Setup

To ensure this script runs automatically every night at 2:00 AM server time, it must be added to the root crontab.

1. Execute `sudo crontab -e`.
2. Add the following line, ensuring the required environment variable is passed to the script:

```
0 2 * * * DB_BACKUP_PASSPHRASE="your_secure_passphrase"
/opt/scripts/backup_db.sh >> /var/log/db_backup.log 2>&1
```

12.4 Disaster Recovery: 3-Step Rollback Protocol

If a deployment introduces critical bugs, data corruption, or downtime, the system must be reverted to the previous day's stable state. The external deployment developer must follow this strict three-step protocol.

Step 1: Revert the Application Code

Halt the current broken application and restore the previous stable git commit.

```
# Navigate to the app directory
cd /var/www/worldmonitor

# Identify the previous working commit
git log --oneline

# Hard reset to the stable commit hash
git reset --hard <STABLE_COMMIT_HASH>

# Install specific dependencies for that commit and rebuild
npm ci
npm run build
```

Step 2: Restore the Database

If database migrations corrupted the schema or data, the database must be restored from the most recent nightly backup.

```
# Navigate to the backup directory
cd /var/backups/postgresql

# Decrypt the most recent backup
gpg --decrypt --batch --passphrase "$DB_BACKUP_PASSPHRASE" -o
restore.sql.gz <MOST_RECENT_BACKUP>.gpg

# Uncompress the backup
gunzip restore.sql.gz

# Drop the corrupted database and recreate a fresh instance
dropdb -U postgres worldmonitor_enterprise
createdb -U postgres worldmonitor_enterprise

# Restore the data
```

```
psql -U postgres -d worldmonitor_enterprise < restore.sql

# Clean up temporary restoration files
rm restore.sql
```

Step 3: Relaunch and Verify

With the codebase and database restored to matching, stable states, the application can be brought back online.

```
# Reload PM2 instances with zero downtime
pm2 reload worldmonitor-enterprise --update-env

# Verify application health
pm2 logs worldmonitor-enterprise --lines 50
```

Upon successful restart, the developer must manually access the portal to verify that the SSO login succeeds and that user preferences are rendering correctly.

Chapter 13: Project Management: Timeline, AI Impact & Costs

13.1 Executive Summary

This chapter details the project management, financial operations (FinOps), and agile planning strategy required to transform the WorldMonitor codebase into the AIB-AXYS Africa Enterprise Intelligence Portal. Executed by a lean team of two human developers augmented by an AI CLI coding agent, the project relies on a zero-cost infrastructure philosophy. This document outlines the development timeline, assesses the impact of AI tooling, and provides a strategic breakdown of API and operational costs.

13.2 Development Timeline & Complexity

The migration and specialization project is structured into four distinct phases, estimated to span a total of 8 to 9 weeks. This accelerated timeline is predicated on the aggressive use of AI coding assistants.

Phase 1: Database & Identity Migration (Weeks 1-2)

- **Complexity:** High
- **Tasks:** Strip Convex from the repository. Initialize PostgreSQL via Prisma or Drizzle. Implement NextAuth/Auth.js and configure the enterprise SSO provider. Create the relational database schemas for user preferences.
- **Milestone:** A user can successfully log in via corporate SSO, and their session is validated against the local PostgreSQL database.

Phase 2: Security & ETL Pipeline Orchestration (Weeks 3-4)

- **Complexity:** Medium
- **Tasks:** Patch all Server-Side Request Forgery (SSRF) vulnerabilities in proxy endpoints. Replace Convex background functions with a serverless orchestrator (e.g., Inngest). Establish the Redis caching layer for instant widget loading.
- **Milestone:** Automated background jobs successfully fetch mock data, persist it to PostgreSQL, and hydrate the Redis cache securely.

Phase 3: African UI Customization & Mapping (Weeks 5-7)

- **Complexity:** High
- **Tasks:** Build the modular, customizable "easy-on-the-eye" dashboard using Zustand. Separate the complex Deck.gl interface to a dedicated route. Integrate the African and Kenyan specific data layers.
- **Milestone:** The dashboard renders dynamic, user-pinned widgets instantly, and the dedicated map view correctly visualizes regional GeoJSON boundaries.

Phase 4: QA, Testing & Production Deployment (Weeks 8-9)

- **Complexity:** Low to Medium
- **Tasks:** Generate Playwright End-to-End tests mocking the authenticated state. Provision the Linux server with PM2 and Nginx. Execute the automated zero-downtime deployment script.

- **Milestone:** The application is live on the corporate network, fully monitored, with daily database backups running autonomously.

13.3 The AI Factor (Speed vs. Limitations)

The integration of an AI CLI agent (e.g., Claude Code or Cursor) radically alters the development velocity for a two-person team. However, strategic awareness of its limitations is required to prevent technical debt.

13.3.1 Acceleration: Where AI Excels

- **Boilerplate & Schema Generation:** The AI will instantly generate Prisma schemas, NextAuth configuration files, and standard CRUD API routes, saving days of manual typing.
- **Test Automation:** The AI is highly proficient at writing Playwright E2E tests, particularly generating the network interception mocks detailed in the QA Playbook.
- **Component Translation:** Converting raw HTML/CSS into React components styled with Tailwind CSS will be executed almost instantly by the agent.

13.3.2 Limitations: Where Human Developers Must Lead

- **Deck.gl and WebGL Complexities:** The AI agent will struggle significantly with spatial mapping logic, custom WebGL shaders, and multi-layered Deck.gl configurations. Human developers must manually debug coordinate projections and layer z-indexing.
- **State Synchronization:** While the AI can write a basic Zustand store, debugging race conditions between optimistic UI updates and asynchronous database writes requires human architectural reasoning.
- **Corporate Network Nuances:** Configuring the server's Nginx reverse proxy to play nicely with specific corporate VPNs, internal DNS, and enterprise SSO SAML certificates requires human oversight.

13.4 FinOps & API Strategy

The overarching directive is to maintain operational costs at or near zero, leveraging existing infrastructure and open-source tooling.

13.4.1 Infrastructure Cost Benefits

- **Next.js Standalone:** By compiling Next.js as a standalone Node.js application deployed via PM2 on existing Linux servers, the project entirely bypasses the premium costs associated with managed serverless platforms (e.g., Vercel Enterprise).
- **Self-Hosted PostgreSQL:** Hosting the database on the same infrastructure eliminates the recurring monthly fees of managed database providers, keeping Capital Expenditure (CapEx) at zero.

13.4.2 Low-Cost Mapping Strategy

The existing codebase may default to Mapbox, which incurs significant costs at high volume.

- **Recommendation:** The engineering team must migrate the map base layer to **MapLibre GL JS** (a free, open-source fork of Mapbox).

- **Tile Providers:** Base map tiles should be sourced from OpenStreetMap (OSM) or the free tier of MapTiler, ensuring mapping costs remain zero for internal corporate use.

13.4.3 Bloomberg Subscription & Financial APIs

Accessing high-quality African financial data without incurring new vendor costs requires leveraging existing corporate assets.

- **Bloomberg Verification:** The team must contact the firm's Bloomberg representative to verify if the current subscription includes the "Desktop API" (DAPI) or "B-PIPE" access. If DAPI is included, developers can route internal requests through a locally hosted Bloomberg terminal service.
- **Open Alternatives:** If Bloomberg API access is restricted or incurs massive fees, the ETL pipelines must rely on the open APIs outlined in the Data Specialization Strategy (e.g., Central Bank of Kenya open feeds, Kiel Institute Africa Monitor, and targeted scraping of public economic portals).

13.5 Project Timeline (Gantt Chart)

The following chart illustrates the end-to-end development schedule.

